

Piccolo manuale **Laravel**

Questo manuale vi aiuterà a rinfrescarvi la memoria nei momenti di buio più totale.

1. Prerequisiti

IMPORTANTE!!!

Non Utilizzate WAMP, XAMP, MAMP, QUACK o qualsiasi altro, munitevi di una Linux Box e assicuratevi di aver installato i seguenti pacchetti/software:

- Git
- Zip
- PHP $\geq$ 8.2
- Composer 2
- npm

Git e **Zip** sono 2 comandi che generalmente vengono installati con qualsiasi ambiente, per maggiori dettagli sui requisiti visitate il link: <https://laravel.com/docs> poi fate click sulla voce di menù "Documentation" ed infine su # Installation > Server Requirements (che trovate poco più sopra del centro della pagina).

Vi lascio i passi essenziali da compiere su una ubuntu **24.04 LTS** che ho personalmente testato i primi giorni di novembre 2024.

1.1. Installare Apache e MySQL

1.1.1. Apache: `$ sudo apt-get install apache2 -y`

1.1.2. MySQL: `$ sudo apt-get install mysql-server -y`

1.2. Configurazione (minimale) di MySQL

```
$ sudo mysql -u root -p
$ < inserire la password dell'utente root del computer >
$ < premere invio perchè l'utente root di mysql non ha passw >
mysql> CREATE USER 'user'@'localhost';
mysql> GRANT ALL ON *.* TO 'user'@'localhost';
mysql> FLUSH PRIVILEGES;
mysql> < uscite con \q >
```

1.3. Installare PHP e i moduli di PHP necessari

```
$ sudo apt-get install php libapache2-mod-php php-mysql -y
```

Servono i moduli per PHP: **zip**, **gd**, **xml**, **curl**, **mbstring**, **merypt** e li installate con ...

```
$ sudo apt-get install php8.3-xxx
```

Esempio:

```
$ sudo apt-get install php8.3-zip -y
```

1.4. Installare Composer

Composer è un gestore di pacchetti per PHP e ci aiuta a gestire le librerie e per installarlo, la Documentazione di Laravel fornisce le indicazioni per tutti i S.O.

Per installare composer su ubuntu, usare il comando:

```
$ sudo apt-get install composer -y
```

I comandi principali sono:

```
$ composer require <nomepackage> [vincolo] [versione]
```

Aggiunge nuove dipendenze aggiungendo librerie al file composer.json

Es: `composer require laravel/laravel ^9.0`

`x.y.z` -> Esattamente la versione x.y.z

`>=x.y.z` -> Versione maggiore o uguale di x.y.z

`^x.y.z` -> Versione vicina a x.y.z (limite alla major version)

`~x.y.z` -> Versione vicina a x.y.z (limite alla release)

```
$ composer update
```

Aggiorna tutte le librerie all'ultima versione disponibile compatibile con quanto indicato in composer.json e al termine crea/aggiorna composer.lock (che contiene le versioni precise di quanto installato).

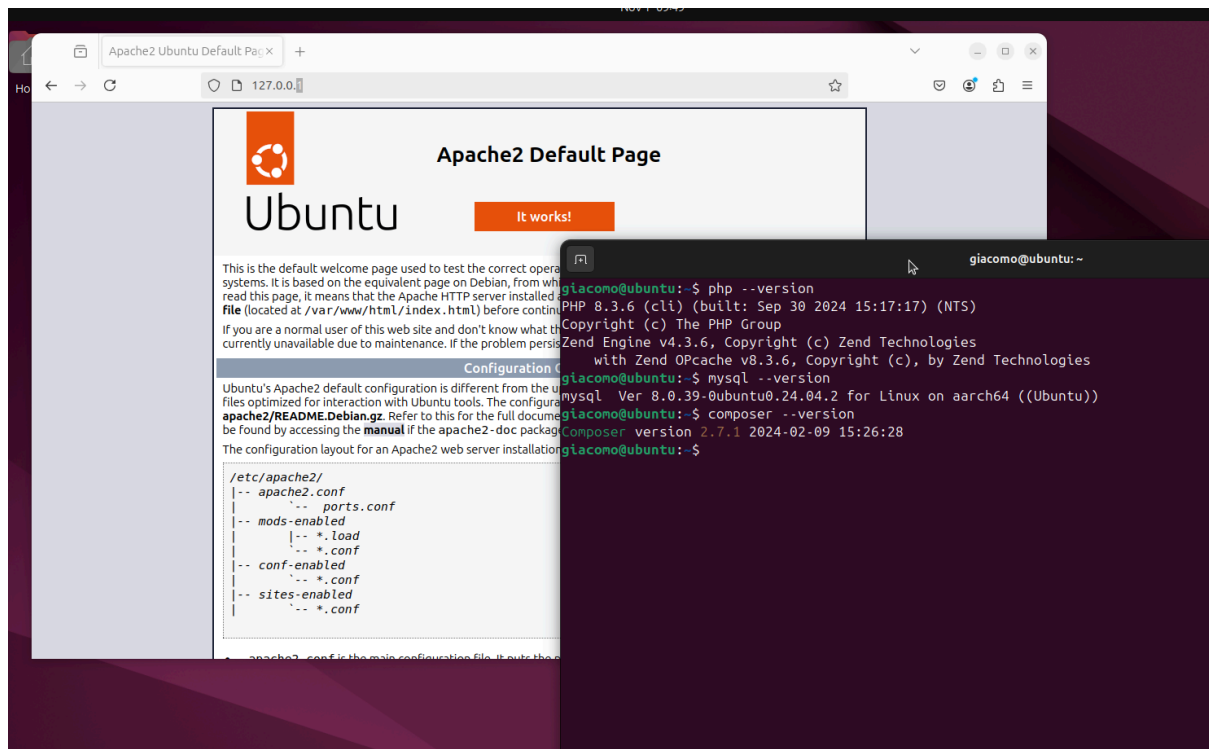
```
$ composer install
```

Installa le versioni specificate in composer.lock (che quindi deve già essere creato con un primo composer update).

```
$ composer clear-cache
```

Pulisce la cache di composer.

1.5. Assicuratevi di aver tutto installato e procedete installando MySQL-Workbench e Visual Studio Code.



Potete installare **Visual Studio Code** scaricandolo da qui:

<https://code.visualstudio.com/download>

Scegliendo il pacchetto adatto ed eseguendo poi il comando:

```
$ sudo dpkg -i <nome-del-pacchetto-scaricato>.deb
```

Per **MySQL Workbench** potete seguire questa guida che copre più situazioni:

<https://phoenixnap.com/kb/mysql-workbench-ubuntu>

In alternativa esiste un **estensione** di Visual Studio Code che riporta **MySQL Workbench** in VSCode stesso (molto utile anche se in Beta)

Potete utilizzare alternative vome **TablePlus** in versione gratuita:

<https://tableplus.com/blog/2019/10/tableplus-linux-installation.html>

Per installare tutto su **Mac** potete utilizzare questa guida:

<https://getgrav.org/blog/macos-sequoia-apache-multiple-php-versions> saltando l'ultimo punto della prima parte: "Test Your Setup with Grav CMS!"

2. Getting Started!

Se durante la creazione di un nuovo progetto ottenete un errore, controllate nella sezione 5 (Troubleshooting) se è elencato il vostro caso.

2.1. Se non lo avete già fatto, come prima cosa create un nuovo database vuoto.

2.2. Creare un nuovo progetto usando uno dei seguenti comandi a seconda se si vuole o meno voler specificare una versione precisa di Laravel:

```
composer create-project laravel/laravel --prefer-dist  
<nome_progetto>  
composer create-project laravel/laravel=9 --prefer-dist  
<nome_progetto>
```

2.2.1. Configurare il progetto:

```
cd <nome_progetto>  
php artisan config:cache
```

A questo punto configurare il file `.env` del nuovo progetto con il driver e i parametri di connessione al database

2.3. Se state installando l'applicazione attraverso l'operazione di *clone* da un repository, dopo il comando "git clone" dovete eseguire:

```
cd <nome_progetto>  
composer install  
cp .env.example .env  
php artisan key:generate  
php artisan config:cache  
php artisan migrate
```

2.4. (Opzionale MA DA FARE ORA) Se il vostro progetto prevede diversi utenti eseguite i seguenti passaggi:

```
composer require laravel/ui  
php artisan ui bootstrap --auth  
npm install --save-dev vite laravel-vite-plugin  
npm install --save-dev @vitejs/plugin-vue  
npm install && npm run build dev
```

Non mi dilungherò sul significato di ogni passaggio, potete trovare maggiori dettagli a questo link: <https://github.com/laravel/ui>

Se avete bisogno di una gestione degli utenti più evoluta (per esempio con i ruoli utente) allora vi consiglio di seguire le indicazioni presenti in [questa discussione su Stack Over Flow](#) oppure potete installare il package [laravel/laravel-permissions](#) che vi garantirà una gestione professionale dei permessi e dei ruoli

2.5. Avviare l'applicazione:

- 2.5.1. Eseguite il comando per riempire il database (Se avete seguito la parte con il login - 2.4 - aprite un nuovo terminale):

```
php artisan migrate
```

- 2.5.2. Eseguire il server locale con il seguente comando:

```
php artisan serve
```

A questo punto visitando il link <http://127.0.0.1:8000>, troverete la pagina predefinita "welcome" del vostro progetto

3. Convenzioni sui nomi

Tabelle del DB

Nome in minuscolo, al plurale con forma “snake” (_ per separare le parole)
es. users, tags, ...

Colonne del DB

Nome in minuscolo al singolare con forma “snake”
es. first_name, last_name, birth_date...

Modelli

Al singolare con forma “cammellizzata” (prima lettera in maiuscolo per separare le parole)
es. User, Tag, ...

Controllers

Al singolare con forma “cammellizzata”
es. UserController, TagController, HelloController ...

Rotte

Al plurale (stesso della tabella)
es. users/ , tags/ , ...

4. Comandi base

Se durante l'esecuzione dei comandi ottenere un errore, controllate nella sezione 4 (Troubleshooting) se è elencato il vostro caso

4.1. Generatori

```
// View
php artisan make:view <nome_della_view>

// Controller
php artisan make:controller <nome_controller_singolare>Controller

// Controller completo di tutte le azioni
php artisan make:controller <nome_controller_singolare>Controller
--resource

// Model
php artisan make:model <nome_model_singolare>

// Model (con migrazione)
php artisan make:model <nome_model_singolare> -m

// Scaffolding (modello + migrazione + controller)
php artisan make:model <nome_model_singolare> -m -c --resource
```

4.2. Migrazioni

```
// Creare una migrazione
php artisan make:migration create_<nome_tabella>_table

php artisan make:migration <modifica>_to_<nome_tabella>_table

// Eseguire le migrazioni
php artisan migrate

// Eseguire il rollback dell'ultima migrazione
php artisan migrate:rollback [--step=n]

// Eseguire il reset del database, cancella tutte le tabelle e
riesegue le migrazioni
php artisan migrate:fresh
```

4.3. Seeder

// Creare un seeder

```
php artisan make:seeder <nome_model_singolare>Seeder
```

// Eseguire tutti i seed

```
php artisan db:seed
```

// Eseguire il seed solo di una specifica classe

```
php artisan db:seed --class=<nome_model_plurale>TableSeeder
```

Potrebbe capitare, durante l'esecuzione dei Seed di ricevere l'errore:

```
Target class [UsersTableSeeder] does not exist.
```

Questo succede perchè al progetto sono stati aggiunti dei file (i seeder appunto) che non riescono ad essere identificati automaticamente.

Per risolvere il problema bisogna eseguire il comando:

```
composer dump-autoload
```

Che andrà ad indicizzare i file aggiunti.

5. Troubleshooting

Errori durante la creazione del progetto su Ubuntu

Dipende ovviamente dal messaggio di errore ma spesso è perchè mancano i seguenti pacchetti, quindi Installateli con i comandi (o cercate i comandi aggiornati):

```
sudo apt-get install php-mbstring
sudo apt-get install php-xml
```

Errori nella lettura del config

Se ricevete degli errori su delle configurazioni che non rispettano quello che avete inserito nel file .env allora dovete eseguire la pulizia della cache del file di config:

```
php artisan config:clear
```

Errore relativo a sass (con molto testo in rosso) durante l'esecuzione di npm

```
npm uninstall --save-dev sass-loader
npm install --save-dev sass-loader@7.1.0
```

E poi di nuovo

```
npm install && npm run dev
```

Il codice JS o CSS non esegue quello che volete o non trova funzioni che avete appena scritto

Eseguire la pulizia della cache delle view:

```
php artisan view:clear
```

Errori che sembrano venire da codice “vecchio” che avete rimosso

Eseguire la pulizia della cache:

```
php artisan cache:clear
```

Errore SQL 42000 durante le migrazioni

Durante l'esecuzione delle migrazioni (generalmente alla prima) appare un errore riguardante una *“Key too long; max length is ...”*

Per risolverlo editate il file `AppServiceProvider.php` e aggiungete in testa:

```
use Illuminate\Support\Facades\Schema;
```

e nella funzione `boot()` inserite la seguente riga di codice:

```
Schema::defaultStringLength(191);
```

A questo punto cancellate le tabelle e rieseguite le migrazioni

6. Materiale

Ecco una serie di link “ufficiali” per imparare Laravel:

<https://www.youtube.com/watch?v=SqTdHCTWqks&t=103s>

<https://laracasts.com/series/laravel-8-from-scratch>

7. Piccolo manuale di Vue-JS

VueJS è un framework di front-end ovvero un framework Javascript che ben si integra all'interno di una applicazione Laravel.

Oggi VueJS è arrivato alla versione 3.x che include anche uno strumento cli per gestire il progetto e non ho ancora indagato la sua integrazione in Laravel, tuttavia questa serie: <https://youtu.be/BZwn47RPiAM> prodotta direttamente da Laracast sembra essere molto promettente.

Il suo utilizzo nella versione 2.x è molto divertente da utilizzare nello sviluppo delle View e vi lascio qui qualche video lezione:

<https://www.youtube.com/watch?v=I9EP8LIVbx4>